Homework 2: Building and Verifying a Tabular RL Agent

COSC 527: Reinforcement Learning

Instructor:	Raffaele Romagnoli
Due date:	Posted on Canvas
Work format:	Individual
Submission:	Source code and a short PDF report

Purpose

This homework is an individual checkpoint for Midterm Project A. You will not be graded on whether you have already learned a successful swing-up controller. Instead, you will build and verify the components on which a correct experiment depends:

- interpretation of the environment and episode outcome;
- conversion of the continuous plant state into a finite state;
- implementation and empirical verification of an ϵ-greedy behavior policy;
- verification of SARSA and Q-learning targets; and
- one preliminary training experiment with diagnostic evidence.

The code developed here may be reused in the midterm project, but this submission must represent your individual work and analysis.

Problem setting

Use the Python Furuta-pendulum environment supplied for Midterm Project A. The continuous state is

$$x = [\phi, \dot{\phi}, \theta, \dot{\theta}]^T,$$

and the default discrete action set applies one of three motor torques:

$$\mathcal{A} = \{-0.02,\ 0,\ +0.02\} \text{ N m}.$$

Use the wrapped pendulum angle

$$\widetilde{\theta} = \text{wrapToPi}(\theta) \in [-\pi, \pi],$$

where $\widetilde{\theta} = 0$ is upright and $|\widetilde{\theta}| = \pi$ is downward. An episode begins near the downward equilibrium. Project success requires the pendulum to remain within

$$|\widetilde{\theta}_t| \leq 20^\circ$$

for five consecutive seconds.

Required work

1. Verify the environment and episode logic (15 points)

Write a short test program that calls `reset()` and `step()`. Report:

- one initial continuous state and its physical interpretation;
- the action index and motor torque for every legal action;
- one sampled transition $(S_t, A_t, R_{t+1}, S_{t+1})$; and

- how the environment distinguishes success, unsafe termination, and time-limit truncation.

Confirm that being outside the upright region does not terminate an episode. Explain why such termination would make the swing-up task impossible.

2. Design and test the state discretization (20 points)

Define bin boundaries for ϕ, $\dot{\phi}$, $\widetilde{\theta}$, and $\dot{\theta}$. Include a table in your report containing:

Variable	Selected range	Number of bins	Reason for the choice
ϕ			
$\dot{\phi}$			
$\widetilde{\theta}$			
$\dot{\theta}$			

Report the resulting number of discrete states and Q-table entries. If the four variables use n_1, n_2, n_3, n_4 bins and there are n_a actions, then

$$|\mathcal{S}| = n_1 n_2 n_3 n_4, \qquad |Q| = |\mathcal{S}| n_a.$$

Test the discretizer using at least eight deliberately selected states, including:

- the upright and downward configurations;
- angles near $-\pi$ and $+\pi$;
- a value on a bin boundary; and
- values below and above the selected velocity ranges.

For each test, report the continuous state and returned table index. Explain how out-of-range values are mapped to valid indices.

3. Verify the ϵ-greedy behavior policy (15 points)

Implement an ϵ-greedy action selector. For the test state $s^\dagger$, set

$$Q(s^\dagger, \cdot) = [1.0,\ 2.0,\ 2.0].$$

Use $\epsilon = 0.20$ and sample 10,000 actions without changing the Q table. Report the empirical frequency of each action.

Your implementation must specify how ties are handled. Compare the observed frequencies with the probabilities implied by your precise implementation. For example, the result depends on whether exploration samples from all actions and whether exploitation selects uniformly between the two greedy actions.

Repeat the experiment with $\epsilon = 0$ and explain the result. Use a fixed random seed and report it.

4. Audit one SARSA and one Q-learning update (20 points)

Create or record one nonterminal transition for which the next-state action values are not all equal. Save the following quantities before performing an update:

$$S_t, \quad A_t, \quad R_{t+1}, \quad S_{t+1}, \quad A_{t+1}, \quad Q_{\text{old}}(S_t, A_t), \quad Q_{\text{old}}(S_{t+1}, \cdot).$$

Using the same stored transition, compute both targets:

$$Y_t^{\text{SARSA}} = R_{t+1} + \gamma Q(S_{t+1}, A_{t+1}),$$

$$Y_t^{\text{Q}} = R_{t+1} + \gamma \max_a Q(S_{t+1}, a).$$

Then compute the corresponding updated values using

$$Q_{\text{new}}(S_t, A_t) = Q_{\text{old}}(S_t, A_t) + \alpha \left[Y_t - Q_{\text{old}}(S_t, A_t) \right].$$

Show the numerical calculation in the report and verify that your program produces the same results. Explain when the two targets are equal and when they are different.

Repeat the calculation for one terminal transition. Demonstrate that the continuation value is zero, regardless of the values stored for the next state.

5. Run one preliminary learning experiment (20 points)

Choose either SARSA or Q-learning and run a preliminary training experiment. The purpose is diagnostic: successful swing-up is not required for this homework. Report all selected values, including the number of episodes, α, γ, exploration schedule, Q-table initialization, random seeds, and maximum episode duration.

Run at least three independent seeds. Include:

- episode return and a moving average;
- episode duration;
- fraction of visited state-action entries; and
- the wrapped pendulum angle from one greedy evaluation episode after training.

During evaluation, freeze the Q table, set $\epsilon = 0$, and do not perform updates. State whether the policy remained near the downward equilibrium, generated a growing swing, reached the upright region, or completed the five-second balancing objective.

6. Diagnostic reflection (10 points)

Answer the following questions in your own words:

(a) Which part of your implementation is most likely to limit learning: the discretization, exploration, reward, update, or episode duration? Support your answer with one result.

(b) Identify one state variable or region that was rarely visited. Would you change its discretization for the project? Why or why not?

(c) Describe one test that detected, or could detect, an implementation error that a training-return plot alone might hide.

Submission

Submit one archive containing:

1. all source files and a short README explaining how to run them;
2. a concise PDF report containing the requested tables, numerical update audit, plots, and written answers; and
3. any saved data needed to reproduce the reported figures.

Suggested report length is 3–5 pages, excluding the generative-AI appendix. The report may reuse figures or analysis in the later midterm-project report, provided the project submission clearly identifies any subsequent changes.

Use of generative AI

Use of generative AI tools is permitted in accordance with the course syllabus. If you use such a tool, include an appendix containing:

- the relevant prompts or questions submitted to the tool;
- a brief description of how the responses influenced your work;
- at least one suggestion that you verified, corrected, modified, or rejected; and
- the portions of the code or report for which the tool provided assistance.

You remain responsible for understanding, verifying, and explaining every part of the submission. Your numerical update audit and empirical policy-frequency test must be generated from your own submitted implementation.

Evaluation criteria

Component	Weight
Environment and episode verification	15%
State discretization and boundary tests	20%
ϵ-greedy policy verification	15%
SARSA and Q-learning update audit	20%
Preliminary learning experiment	20%
Diagnostic reflection and clarity	10%

Learning a successful swing-up controller is not required for full credit. Correct implementation, reproducibility, meaningful verification, and careful interpretation are the primary grading criteria.